

```

# =====
# BLOB Principe - SAUT 3.5 : INVARIANTS D ECHELLE GRANDE SIMULATION
# =====
#
# Objectif : verifier si certaines observables convergent vers des valeurs
# limites stables a grand N, candidates pour des invariants d echelle Blob.
#
# Methode :
#   1 simulation Blob 2.0 v2 jusqu a 1M pas (sur 1 seed pour rapidite)
#   Snapshots detailles a 8 paliers : 10k, 50k, 100k, 200k, 400k, 600k, 800k, 1M
#   A chaque palier : calcul complet PHI, PSI, Q1, Q3, D, r3, et ratios candidats
#
# Tests d invariance :
#   I1 - PHI/n_tetra converge ?
#   I2 - PSI converge ?
#   I3 - Q1/PSI converge (point de fusion global<->local) ?
#   I4 - log(n_tetra)/log(N) converge (exposant scaling) ?
#   I5 - r3 sature ? (deja [M] mais reverif)
#   I6 - (tau/beta) * r3 converge vers kappa_0 = 8 ?
# =====

```

```
import sys
```

```
import os
```

```

def auto_check():
    print("=" * 70)
    print("  AUTO-TEST CORRUPTION CARACTERES")
    print("=" * 70)
    try:
        if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv
            source_file = sys.argv[0]
        else:
            source_file = __file__ if '__file__' in globals() else None
        if source_file and os.path.exists(source_file):
            with open(source_file, 'r', encoding='utf-8') as f:
                content = f.read()
            dangerous = {chr(8216): "U+2018", chr(8217): "U+2019",
                chr(8220): "U+201C", chr(8221): "U+201D",
                chr(8211): "U+2013", chr(8212): "U+2014"}
            found = [(name, content.count(c)) for c, name in dangerous.items() if
            if found:
                print("ERREUR caracteres corrompus :", found)
                return False
            print("OK : pas de corruption")

```

```

        else:
            print("Note : auto-test partiel")
    except Exception as e:
        print("Note : " + str(e))
    print("=" * 70)
    return True

if not auto_check():
    sys.exit(1)

import numpy as np
import time
import json
import gc
from collections import Counter

KAPPA_0 = 8.0
BETA = 0.30
TAU = 0.50
P_MITOSE = 0.001
EPS_DECAL = 0.05

OUTPUT_DIR = '/content/blob_saut35_results'
os.makedirs(OUTPUT_DIR, exist_ok=True)

class BlobUniverse:
    def __init__(self, seed=0):
        self.rng = np.random.default_rng(seed)
        self.seed = seed
        self.positions = [np.array([0.5, 0.5, 0.5])]
        self.adj = [set()]
        self.edge_list = []
        self.deg = np.array([0], dtype=np.int32)
        self.n_tri = 0
        self.n_tetra = 0
        self.S_curr = 0.0
        self.Q3_dissipation = 0.0
        self.n_mitoses = 0
        self.n_swaps_accepted = 0
        self.first_edge_step = None
        self.first_triangle_step = None
        self.first_tetrahedron_step = None

    @property
    def N(self):
        return len(self.positions)

```

```

@property
def E(self):
    return len(self.edge_list)

def _compute_D(self):
    if self.N == 0:
        return 0.0
    mu = float(self.deg.mean())
    if mu == 0:
        return 0.0
    return float(self.deg.std()) / mu

def _compute_Q1(self):
    if self.N == 0:
        return 0.0
    c = Counter(self.deg.tolist())
    ent = 0.0
    for cnt in c.values():
        p = cnt / self.N
        if p > 0:
            ent -= p * np.log(p)
    return float(ent)

def _rips_radius(self):
    if self.N <= 1:
        return 0.5
    c_3 = 4 * np.pi / 3
    return 1.5 * (KAPPA_0 / (c_3 * self.N)) ** (1.0/3)

def _count_tetra_through_edge(self, u, v):
    common = self.adj[u] & self.adj[v]
    if len(common) < 2:
        return 0
    cl = list(common)
    count = 0
    for i, w in enumerate(cl):
        adj_w = self.adj[w]
        for x in cl[i+1:]:
            if x in adj_w:
                count += 1
    return count

def _list_tetrahedra(self):
    result = []
    seen = set()
    for u, v in self.edge_list:

```

```

        common = self.adj[u] & self.adj[v]
        if len(common) < 2:
            continue
        cl = sorted(common)
        for i, w in enumerate(cl):
            for x in cl[i+1:]:
                if x in self.adj[w]:
                    key = tuple(sorted([u, v, w, x]))
                    if key not in seen:
                        seen.add(key)
                        result.append(key)
    return result

def compute_PHI(self):
    if self.n_tetra == 0:
        return 0.0
    tetras = self._list_tetrahedra()
    if not tetras:
        return 0.0
    edge_to_tetras = {}
    for tet in tetras:
        for i in range(4):
            for j in range(i+1, 4):
                e = (tet[i], tet[j])
                edge_to_tetras.setdefault(e, []).append(tet)
    neighbors_count = {tet: set() for tet in tetras}
    for e, tlist in edge_to_tetras.items():
        if len(tlist) > 1:
            for i, t1 in enumerate(tlist):
                for t2 in tlist[i+1:]:
                    neighbors_count[t1].add(t2)
                    neighbors_count[t2].add(t1)
    total = sum(len(v) for v in neighbors_count.values())
    return float(total) / len(tetras)

def compute_PSI(self):
    if self.N == 0:
        return 0.0
    total_ent = 0.0
    n_active = 0
    for v in range(self.N):
        neighbors = self.adj[v]
        if not neighbors:
            continue
        n_active += 1
        local_degs = [self.deg[u] for u in neighbors]
        counts = Counter(local_degs)

```

```

        total = len(local_degs)
        ent = 0.0
        for c in counts.values():
            p = c / total
            if p > 0:
                ent -= p * np.log(p)
        total_ent += ent
    return float(total_ent / n_active) if n_active > 0 else 0.0

def mitose(self):
    if self.N == 0:
        return
    parent = int(self.rng.integers(self.N))
    parent_pos = self.positions[parent]
    r_p = self._rips_radius()
    decal = self.rng.normal(0, EPS_DECAL * r_p, size=3)
    new_pos = parent_pos + decal
    self.positions.append(new_pos)
    self.adj.append(set())
    self.deg = np.append(self.deg, 0)
    self.n_mitoses += 1

def step(self, current_step):
    if self.rng.random() < P_MITOSE:
        self.mitose()
        return False
    if self.N < 2:
        return False
    if self.E == 0:
        return self._step_add_only(current_step)
    E_target = int(KAPPA_0 * self.N / 2)
    if self.E < E_target:
        deficit = (E_target - self.E) / max(E_target, 1)
        if self.rng.random() < deficit:
            return self._step_add_only(current_step)
    return self._step_swap(current_step)

def _step_add_only(self, current_step):
    u = int(self.rng.integers(self.N))
    v = int(self.rng.integers(self.N))
    if u == v or v in self.adj[u]:
        return False
    u, v = min(u, v), max(u, v)
    dt_add = len(self.adj[u] & self.adj[v])
    self.adj[u].add(v)
    self.adj[v].add(u)
    self.deg[u] += 1

```

```

self.deg[v] += 1
new_n_tri = self.n_tri + dt_add
dtetra_add = self._count_tetra_through_edge(u, v)
new_S = np.log(1 + new_n_tri) - BETA * self._compute_D()
delta_S = new_S - self.S_curr
log_alpha = self.N * delta_S / TAU
accepted = (log_alpha >= 0) or (self.rng.random() < np.exp(max(log_alpha,
if accepted:
    self.edge_list.append((u, v))
    self.n_tri = new_n_tri
    self.n_tetra += dtetra_add
    self.S_curr = new_S
    self.n_swaps_accepted += 1
    self.Q3_dissipation += abs(delta_S)
    if self.first_edge_step is None and self.E > 0:
        self.first_edge_step = current_step
    if self.first_triangle_step is None and self.n_tri > 0:
        self.first_triangle_step = current_step
    if self.first_tetrahedron_step is None and self.n_tetra > 0:
        self.first_tetrahedron_step = current_step
    return True
else:
    self.adj[u].discard(v)
    self.adj[v].discard(u)
    self.deg[u] -= 1
    self.deg[v] -= 1
    return False

def _step_swap(self, current_step):
    idx = int(self.rng.integers(self.E))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)
    if (u_add, v_add) == (u_rm, v_rm):
        return False
    dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
    dtetra_rm = self._count_tetra_through_edge(u_rm, v_rm)
    self.adj[u_rm].discard(v_rm)
    self.adj[v_rm].discard(u_rm)
    self.deg[u_rm] -= 1
    self.deg[v_rm] -= 1
    dt_add = len(self.adj[u_add] & self.adj[v_add])
    self.adj[u_add].add(v_add)
    self.adj[v_add].add(u_add)

```

```

self.deg[u_add] += 1
self.deg[v_add] += 1
dtetra_add = self._count_tetra_through_edge(u_add, v_add)
new_n_tri = self.n_tri - dt_rm + dt_add
if new_n_tri < 0:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False
new_S = np.log(1 + new_n_tri) - BETA * self._compute_D()
delta_S = new_S - self.S_curr
log_alpha = self.N * delta_S / TAU
accepted = (log_alpha >= 0) or (self.rng.random() < np.exp(max(log_alpha,
if accepted:
    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.n_tetra += (dtetra_add - dtetra_rm)
    self.S_curr = new_S
    self.n_swaps_accepted += 1
    self.Q3_dissipation += abs(delta_S)
    if self.first_triangle_step is None and self.n_tri > 0:
        self.first_triangle_step = current_step
    if self.first_tetrahedron_step is None and self.n_tetra > 0:
        self.first_tetrahedron_step = current_step
    return True
else:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False

```

```

def measure_full_state(sim, t_start):
    """Calcul lourd : PHI, PSI, Q1, et tous derives. A faire seulement aux palier
    t0 = time.time()
    PHI = sim.compute_PHI()
    PSI = sim.compute_PSI()

```

```

Q1 = sim._compute_Q1()
D = sim._compute_D()

measure_time = time.time() - t0

state = {
    'N': sim.N,
    'E': sim.E,
    'n_tri': sim.n_tri,
    'n_tetra': sim.n_tetra,
    'D': D,
    'Q1': Q1,
    'PHI': PHI,
    'PSI': PSI,
    'Q3': sim.Q3_dissipation,
    'r3': sim.n_tetra / sim.n_tri if sim.n_tri > 0 else 0.0,
    'PHI_over_ntetra': PHI / sim.n_tetra if sim.n_tetra > 0 else 0.0,
    'Q1_over_PSI': Q1 / PSI if PSI > 0 else 0.0,
    'PSI_over_Q1': PSI / Q1 if Q1 > 0 else 0.0,
    'log_ntetra_over_log_N': np.log(sim.n_tetra) / np.log(sim.N) if sim.N > 1
    'tau_beta_r3': (TAU/BETA) * (sim.n_tetra / sim.n_tri) if sim.n_tri > 0 el
    'time_total_s': time.time() - t_start,
    'measure_time_s': measure_time,
}
return state

# =====
# MAIN
# =====
print('')
print('=' * 75)
print('  BLOB - SAUT 3.5 - INVARIANTS D ECHELLE GRANDE SIMULATION')
print('=' * 75)

TOTAL_STEPS = int(os.environ.get('BLOB_TOTAL_STEPS', 1000000))
SEED = int(os.environ.get('BLOB_SEED', 0))

# Paliers : exponentiels pour bien voir convergence ou non
PALIERS = [10000, 50000, 100000, 200000, 400000, 600000, 800000, 1000000]
PALIERS = [p for p in PALIERS if p <= TOTAL_STEPS]

print(f'  Configuration : seed={SEED}, total_steps={TOTAL_STEPS}')
print(f'  Paliers : {PALIERS}')
print(f'  Duree estimee : ~5-10 minutes (PHI cher a grand N)')
print('=' * 75)

```

```

t_start = time.time()
sim = BlobUniverse(seed=SEED)
snapshots = []

next_palier_idx = 0
last_progress = 0

for step in range(TOTAL_STEPS):
    sim.step(step + 1)

    # Progress simple
    if step - last_progress >= 50000:
        elapsed = time.time() - t_start
        print(f'  step={step+1:>8} | N={sim.N:>5} E={sim.E:>6} n_tetra={sim.n_tet
        last_progress = step

    # Palier : mesure complete
    if next_palier_idx < len(PALIERS) and (step + 1) >= PALIERS[next_palier_idx]:
        palier = PALIERS[next_palier_idx]
        print(f'')
        print(f'  === PALIER {palier} ===', flush=True)
        state = measure_full_state(sim, t_start)
        state['palier'] = palier
        state['step'] = step + 1
        snapshots.append(state)
        print(f'    N={state["N"]} n_tetra={state["n_tetra"]}', flush=True)
        print(f'    PHI={state["PHI"]:.3f} PSI={state["PSI"]:.4f} Q1={state["Q1"]
        print(f'    PHI/ntetra={state["PHI_over_ntetra"]:.4f} Q1/PSI={state["Q1_o
        print(f'    r3={state["r3"]:.4f} (tau/beta)*r3={state["tau_beta_r3"]:.4f}
        print(f'    log(ntetra)/log(N)={state["log_ntetra_over_log_N"]:.4f}', flu
        print(f'    measure_time={state["measure_time_s"]:.1f}s', flush=True)
        next_palier_idx += 1

total_time = time.time() - t_start
print(f'')
print(f'=' * 75)
print(f'  CAMPAGNE TERMINEE en {total_time/60:.1f} min')
print(f'=' * 75)

# Recount final pour coherence
tri_recount = sum(len(sim.adj[u] & sim.adj[v]) for u, v in sim.edge_list) // 3
coherent = (sim.n_tri == tri_recount)
print(f'  Coherence finale : tri increment={sim.n_tri} recount={tri_recount} -> {

# =====
# ANALYSE INVARIANCE
# =====

```

```

print(f'')
print(f'=' * 75)
print(f'  ANALYSE D INVARIANCE PAR OBSERVABLE')
print(f'=' * 75)

if len(snapshots) >= 3:
    # Pour chaque observable cle, regarder evolution
    obs_to_test = [
        ('PHI', 'PHI'),
        ('PSI', 'PSI'),
        ('Q1', 'Q1'),
        ('PHI/ntetra', 'PHI_over_ntetra'),
        ('Q1/PSI', 'Q1_over_PSI'),
        ('log(ntetra)/log(N)', 'log_ntetra_over_log_N'),
        ('r3', 'r3'),
        ('(tau/beta)*r3', 'tau_beta_r3'),
        ('D', 'D'),
    ]

    # Header
    palier_strs = [f'p{p//1000}k' for p in PALIERS[:len(snapshots)]]
    print(f'  {"Observable":25s}  ' + '  '.join([f'{p:>10s}' for p in palier_strs
    print('  ' + '-' * (25 + 2 + 12*len(snapshots) + 25))

    invariant_candidates = []

    for label, key in obs_to_test:
        vals = [s[key] for s in snapshots]
        line = f'  {label:25s}  '
        for v in vals:
            line += f'{v:>10.4f}  '
        # Evolution totale (premier vs dernier)
        if vals[0] != 0:
            evol = (vals[-1] - vals[0]) / abs(vals[0]) * 100
        else:
            evol = float('nan')
        # CV sur 3 derniers paliers (tendance asymptotique)
        if len(vals) >= 3:
            tail = vals[-3:]
            mean_tail = np.mean(tail)
            std_tail = np.std(tail)
            cv_tail = (std_tail / abs(mean_tail) * 100) if mean_tail != 0 else 0
        else:
            cv_tail = 0
        line += f'  {evol:>+8.1f}%  {cv_tail:>5.2f}%'
        if cv_tail < 2.0:
            line += '  ** INVARIANT **'

```

```

        invariant_candidates.append((label, key, vals, cv_tail))
    elif cv_tail < 5.0:
        line += ' * stabilisant *'
    print(line)

print(f'')
print(f'=' * 75)
if invariant_candidates:
    print(f' CANDIDATS INVARIANTS (CV < 2% sur 3 derniers paliers) :')
    for label, key, vals, cv in invariant_candidates:
        print(f' {label} : valeur asymptotique ~ {np.mean(vals[-3:]):.4f}')
else:
    print(f' Aucun candidat invariant strict (CV < 2%) sur les 3 derniers pa
print(f'=' * 75)

# Sauvegarde
report = {
    'config': {
        'seed': SEED,
        'total_steps': TOTAL_STEPS,
        'paliers': PALIERS,
        'tau': TAU, 'beta': BETA, 'kappa_0': KAPPA_0,
        'total_time_s': total_time,
    },
    'snapshots': snapshots,
    'final_coherent': coherent,
    'tri_recount': tri_recount,
}

report_path = OUTPUT_DIR + '/saut35_RAPPORT.json'
with open(report_path, 'w') as f:
    json.dump(report, f, indent=2)
print(f'')
print(f' Rapport sauvegarde : {report_path}')
print(f' Taille : {os.path.getsize(report_path)} octets')

```